TASK 3 – Flask Mini Project & Backend Fundamentals

Project Title:

Notes Web Application using Flask

1. Objective

The objective of this project is to learn backend web development fundamentals using Python and Flask. The project builds a fully functional Notes web application that demonstrates routing, form handling, templating, and basic CRUD data operations through a clean Bootstrap-based UI.

2. Technologies Used

- Python
- Flask Framework
- Jinja2 Templating Engine
- Bootstrap 5 (CDN)
- JSON (file-based data storage)
- HTML / CSS

3. Features

- Full CRUD – Create, Read, Update, Delete notes
- Flask routing with GET and POST methods
- Form handling with server-side validation
- Flash messages for user feedback (success, error, info)
- Jinja2 template inheritance using a base layout
- Notes stored in a JSON file (no database required)
- Responsive UI using Bootstrap 5 card components
- Delete confirmation using JavaScript confirm dialog

4. Code Explanation

The application is built using Flask, a lightweight Python web framework. Each route in app.py handles a specific page or action. The home route (/) loads all notes from a JSON file and passes them to the index template for display. The /add route handles both GET (show form) and POST (save note) requests. Server-side validation checks that title and content are not empty before saving. The /edit/<id> route works similarly, loading the existing note and updating it on form submission. The /delete/<id> route uses a POST-only form to safely remove a note. Flash messages are used throughout to give the user clear feedback. All note data is stored in notes.json using Python's built-in json module, making the app easy to run without any external database.

5. Source Code

app.py

```
from flask import Flask, render_template, request, redirect, url_for, flash
```

```
import json, os
from datetime import datetime

app = Flask(__name__)
app.secret_key = "mysecretkey123"
NOTES_FILE = "notes.json"

def load_notes():
    if os.path.exists(NOTES_FILE):
        with open(NOTES_FILE, 'r') as f:
            return json.load(f)
    return []

def save_notes(notes):
    with open(NOTES_FILE, 'w') as f:
        json.dump(notes, f, indent=2)

@app.route('/')
def index():
    notes = load_notes()
    return render_template('index.html', notes=notes)

@app.route('/add', methods=['GET', 'POST'])
def add_note():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        content = request.form.get('content', '').strip()
        if not title or not content:
            flash('Title and content are required!', 'danger')
            return render_template('add_note.html')
        notes = load_notes()
        existing_ids = [n['id'] for n in notes]
        new_id = 1
        while new_id in existing_ids: new_id += 1
        notes.append({'id': new_id, 'title': title, 'content': content,
            'created_at': datetime.now().strftime('%Y-%m-%d %H:%M')})
        save_notes(notes)
        flash('Note added successfully!', 'success')
        return redirect(url_for('index'))
    return render_template('add_note.html')

@app.route('/edit/<int:note_id>', methods=['GET', 'POST'])
def edit_note(note_id):
    notes = load_notes()
    note = next((n for n in notes if n['id'] == note_id), None)
    if not note:
        flash('Note not found!', 'danger')
        return redirect(url_for('index'))
    if request.method == 'POST':
        note['title'] = request.form.get('title','').strip()
        note['content'] = request.form.get('content','').strip()
        note['updated_at'] = datetime.now().strftime('%Y-%m-%d %H:%M')
        save_notes(notes)
        flash('Note updated!', 'success')
        return redirect(url_for('index'))
    return render_template('edit_note.html', note=note)

@app.route('/delete/<int:note_id>', methods=['POST'])
def delete_note(note_id):
```

```
    notes = [n for n in load_notes() if n['id'] != note_id]
    save_notes(notes)
    flash('Note deleted.', 'info')
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)
```

6. Output Screenshots

- **PROJECT FOLDER STRUCTURE**

The project folder contains: static/, templates/ (add_note.html, base.html, edit_note.html, index.html, view_note.html), app.py, notes.json, README.md, requirements.txt

- **HOME PAGE – ALL NOTES**

```
def load_notes():
    if os.path.exists(NOTES_FILE):
        with open(NOTES_FILE, "r") as f:
            return json.load(f)
    return []

def save_notes(notes):
    with open(NOTES_FILE, "w") as f:
        json.dump(notes, f, indent=2)

@app.route("/")
def index():
    notes = load_notes()
    return render_template("index.html", notes=notes)
```

- **ADD NOTE FORM**

```
@app.route("/add", methods=["GET", "POST"])
def add_note():
    if request.method == "POST":
        title = request.form.get("title", "").strip()
        content = request.form.get("content", "").strip()
        if not title or not content:
            flash("Title and content are required!", "danger")
            return render_template("add_note.html")
        notes = load_notes()
        new_note = {
            "id": len(notes) + 1 if notes else 1,
            "title": title,
            "content": content,
            "created_at": datetime.now().strftime("%Y-%m-%d %H:%M")
        }
        # Make sure IDs are unique even after deletions
        existing_ids = [n["id"] for n in notes]
        new_id = 1
        while new_id in existing_ids:
            new_id += 1
        new_note["id"] = new_id
        notes.append(new_note)
        save_notes(notes)
        flash("Note added successfully!", "success")
        return redirect(url_for("index"))
    return render_template("add_note.html")
```

- **EDIT NOTE FORM**

```
@app.route("/edit/<int:note_id>", methods=["GET", "POST"])
def edit_note(note_id):
    notes = load_notes()
    note = next((n for n in notes if n["id"] == note_id), None)
    if note is None:
        flash("Note not found!", "danger")
        return redirect(url_for("index"))
    if request.method == "POST":
        title = request.form.get("title", "").strip()
        content = request.form.get("content", "").strip()
        if not title or not content:
            flash("Title and content are required!", "danger")
            return render_template("edit_note.html", note=note)
        note["title"] = title
        note["content"] = content
        note["updated_at"] = datetime.now().strftime("%Y-%m-%d %H:%M")
        save_notes(notes)
        flash("Note updated!", "success")
        return redirect(url_for("index"))
    return render_template("edit_note.html", note=note)
```

- **FLASH MESSAGE OUTPUT**

Flash messages appear after each action: 'Note added successfully!' (success/green), 'Note updated!' (success/green), 'Note deleted.' (info/blue), 'Title and content are required!' (danger/red).

7. Conclusion

This project helped in understanding Flask routing, Jinja2 template inheritance, GET and POST form handling, flash messaging, and basic file-based data storage using JSON. Building a full CRUD application from scratch gave practical experience in how backend web development works end to end.

8. GitHub Repo link:

https://github.com/B-Karthik13/Alfido-Tech-Internship/tree/main/TASK-3%20Flask%20Project

9. LinkedIn:

Flask Mini Project – Notes Web Application using Python & Bootstrap